

Unit - III

* Creating table with relationship implementation of key and integrity :-

For creating a table :- For creating a table with relationship we need to use following key constraints

1) PRIMARY KEY

2) FOREIGN KEY (referential integrity constraint)

Ex:-

create student table (

Sid number (10), PRIMARY KEY,

Sname varchar (10),

Sbranch varchar (10)

);

insert into student values (1, 'X', 'AIML');

insert into student values (2, 'Y', 'AI');

insert into student values (3, 'Z', 'DS');

select * from student;

Output:-

SID	Sname	SBranch
1	X	AIML
2	Y	AI
3	Z	DS

create table games (

gid number(10) PRIMARYKEY,

gname varchar(15),

sid FOREIGNKEY(10) reference TO student (sid)

);

insert into games values (1, 'cricket', 1);

insert into games values (2, 'chess', 2);

insert into games values (3, 'Football', 2);

select sid, sname

select sid, sname from student where student[sid=10]

game.sid

Output:-

sid	sname
1	X
2	Y

Query:- retrieve the student names belongs to depart of

AI

de

select sname from student where sbranch = AI

Sname
y

select gname from ~~students~~^{games} where gname = "chess";

output:-

gname
chess

select Sid, Sname from student;

output:-

SID	Sname
1	X
2	Y
3	Z

select gid, gname from games;

gid	gname
11	cricket
22	chess
33	Football

SID	Sname
1	X
2	Y
3	Z

Query:- retrieve the names of students who are players

Inner Query:-

select * from games where student.Sid = games.gid

gid	gname
11	Cricket
22	chess
33	Football

Outer Query:-

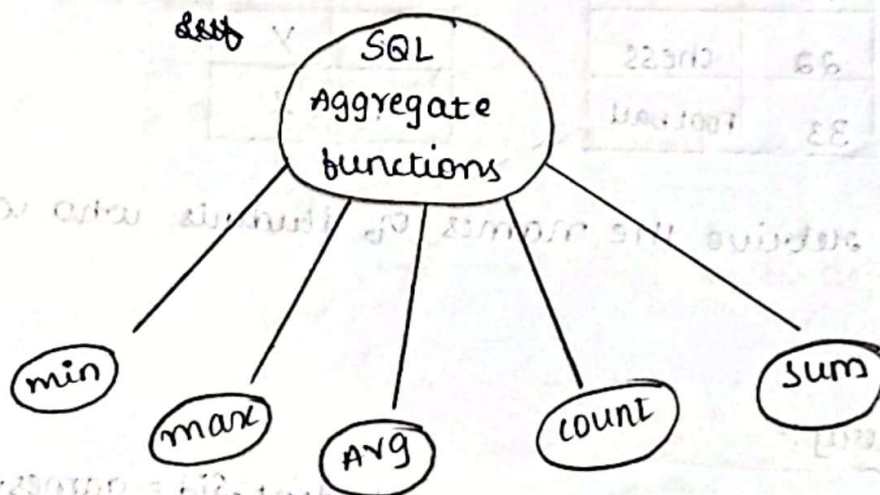
Select Sname from students where exist (Select * from
Select * from games where student id = games id

Output:-

sname
x
y

* Grouping, aggregation, Ordering :-

⇒ aggregation :- Aggregate functions are used to perform the calculations on multiple rows of a single column of a table. it returns a single value by summarizing the data.



Function	description
min	returns the minimum value of a column
max	returns the maximum value of a column
Avg	returns the average of a column
sum	returns the sum of numeric values in a given column
count	returns the count total number of values.
count(*)	returns the no. of rows in a table

Example:-

Create Table:-

Create an Employee table (

 eid number(10) PRIMARY KEY,

 ename Varchar(15),

 salary number(15)

);

insert into Employee values (1, 'X', 50,000);

insert into Employee values (2, 'Y', 20,000);

insert into Employee values (3, 'Z', 30,000);

select * from Employee;

eid	ename	esalary
1	X	50,000
2	Y	20,000
3	Z	30,000

min:-

Query:- Retrieve the minimum salary of an employee

select MIN(esalary) from employee;

esalary
20,000

MAX:-

Query:- Retrieve the maximum salary of an employee

select MAX(esalary) from Employee;

esalary
50,000

Sum:-

Query:- Retrieve the total salaries of all employees in a company.

select SUM(esalary) from Employee;

esalary
100000

AVG:-

Sele

Coun

Que

delec

outp

Coun

Que

del

* Side

date

Syn

Sele

Sele

AVG:- retrieve the average salaries of all Employees.

select AVG(salary) from Employee;

salary
33.333

Count:-

Query:- retrieve count of all Employees.

select COUNT(salary) from Employee;

Output:-

salary
3

only salary employee
will be counted

Count(*):-

Query:- retrieve count* Employee details.

select COUNT(*) from Employee;

salary
3

all Employees will be
counted

* Ordering:- By using Order, we will sort the column data in ascending & descending Order.

Syntax:-

select column1, column2, --- from table name (where
table condition) Order by column1, column2 --
(ascending - descending);

select * from Employee;

O/P:-

eid	ename	esalary
1	x	50,000
2	y	20,000
3	z	30,000

Or Syntax:-

select column from table order by column asc/desc;

⇒ order by sorts the records in 2 ways

- (1) ascending order
- (2) descending order.

⇒ By default order by sorts in ascending order

Order by Ascending:-

select * from Employee order by esalary;

eid	ename	esalary
2	y	20000
3	z	30000
1	x	50000

Or
(Syntax) → select * from Employee order by esalary asc;

Order by descending :-

select * from employee order by salary desc;

eid	ename	salary
1	X	50000
3	Z	30000
2	Y	20000

* Implementation of Views :-

view:- view is a virtual table which is executed by query from an existing table

syntax for creation of a view:-

create VIEW viewname as

select column1, column2 - - -

from tablename

where condition;

⇒ view is a subtable of a parent table or another table

⇒ a view contains rows and columns like real or parent table

creation of view:-

Example:-

```
create table student (Sno number (10),  
                      Sname varchar2 (10),  
                      Sbranch varchar (15),  
                      Smarks number (10));
```

```
insert into student values (1, 'Sai', 'cse', 90);
```

```
insert into student values (2, 'ram', 'ece', 80);
```

```
insert into student values (3, 'Sita', 'eee', 95);
```

```
insert into student values (5, 'Krishna', 'cse', 94);
```

```
select * from student;
```

Sno	Sname	Sbranch	Smarks
1	dai	cse	90
2	ram	ece	80
3	Sita	eee	95
5	Krishna	cse	94

creation of view:-

```
create VIEW cse as
```

```
select * from student
```

```
where Sbranch = 'cse';
```

```
select * from 'cse';
```

output:-

Sno	Sname	Sbranch	Smarks
1	Sai	cse	90
5	Krishna	Cse	94

∴ view is created from the existed table i.e. created a cse student details as a view from students table

Types of views:-

- i. updatable view
- ii. non-updatable view.

⇒ updatable view:- In an updatable view there must be number of fields and data types, constraints must be same as real table.

⇒ student (real table):-

Sno	Sname	Sbranch	Smarks
1	Sai	cse	90
2	ram	ece	80
3	dita	eee	95
5	Krishna	Cse	94

⇒ student cse (view table):-

Sno	Sname	SBranch	Smarch
1	dai	cse	90
2	ram	Ece	80
3	Krishna	cse	94

* updatation on real table and reflection in view table:-

Insert a row into real table:-

insert into student values (4, 'raju', 'cse', 96);

select * from student;

Sno	Sname	Sbranch	Smarchs
1	dai	cse	90
2	ram	ece	80
3	dita	eee	95
4	raju	cse	96
5	Krishna	cse	94

output:-

Sno:4 details are updated in real table

check the view table update:-

select * from cse;

Sno
1
5
4

update

delete

delete

Sno
1
4

No-

tal

em

ei

2

1

1

1

1

1

1

Sno	Sname	Sbranch	Smark
1	dai	cse	90
5	Krishna	cse	94
4	raju	cse	96

updation on view table and reflection on real table:

deletion from cse table:-

deletion from cse where Sname = 'Krishna';

Sno	Sname	Sbranch	Smarks
1	dai	cse	90
4	raju	cse	96

No-updatable view:-

If the number of fields in real table and view table then updation is not possible * *

employee table:-

eid	ename	esalary
1	X	50000
2	Y	20000
3	Z	30000.

create a view:

create VIEW emp as

select eid, ename

from employee

select * from exp;

eid	ename
1	x
2	y
3	z

⇒ updatation in real table and reflection in view table:

insert into employee values (4, 'P', 10000);

select * from employees;

eid	ename	esalary
1	* x	50000
2	y	20000
3	z	30000
4	P	10000

eid:4 details are updatation in real table.

check the updatation in view table :-

select * from exp = emp;

eid	ename
1	x
2	y
3	z
4	p

eid=4 details are updated in view table

→ updatation in view table reflection in parent table:-

insert into Emp values (5, 'a');

select * from Emp;

eid	ename
1	x
2	y
3	z
4	p
5	a

→ eid:5 details are updated in view table.

check the updatation in real table:-

select * from Employee;

eid	ename	esalary
1	x	50000
2	y	20000
3	z	30000
4	p	10000
5	a	

→ real table is not updated

Sub Queries and Nested Queries

Nested Query: Nested Query is a Query Embedded in it that has another Query Embedded in it.

The Embedded Query is called Sub Query. In nested Query, a Query is returned inside a Query. The results of the inner Query is used in Exhibition of outer Query.

Sub Query is closed in the Parenthesis.

There are mainly two types of nested Queries:

- 1) Independent nested Queries.
- 2) Correlated nested Queries.

⇒ Independent nested Queries:-

In these Queries Execution starts from inner most Query to outer most Query. The Execution of inner most Query is independent on Execution of outer not.

Ex:- in, not in

⇒ Correlated nested Queries:-

In this the output Query depends on row which is being currently Executed in outer Query.

Ex:-

→ In

* Join

8

11

11

11

→ 11

11

11

11

11

11

11

11

11

11

11

11

11

11

11

11

11

11

11

→ Implementation of Join :-

* Join :- Joins are used to continue the rows from 2 or more tables. Based on a related column between them.

Joins are introduced to overcome the limitation of cartesian product.

→ Types of Join :-

i. Inner Join :- returns the records that have matching value in both tables.

ii. Outer Join :- Outer join is sub-divided into 3 types

⇒ Left Outer Join :- Returns all the records from the left table and records from right table.

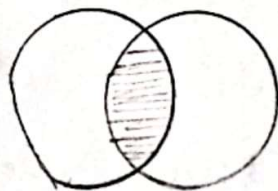
⇒ Right Outer Join :- Returns all the records from the right table and match records from left table

iii. Full Outer Join :- returns all the records where there is a match in left or right table.

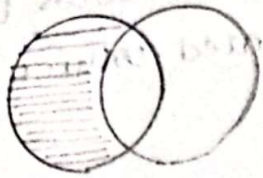
→ Diagrammatic representation of join :-

⇒ Inner Join :-

returns common or similar tuples.

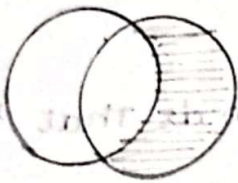


⇒ Left outer join :-



returns left table records
and matching records of
right table

⇒ Right outer join :-



return right table records, and
matching records of left table

⇒ Full outer join :-



returns left table records right
table records matching records
of both.

Student table :-

Sid	Sname	Cid
1	Robo	10
2	dana	20
3	chitti	30

course table

cid	cname
10	C
30	Java
40	C++

Inner join: Inner join returns records of both tables which were matched.

Syntax:

```
select column names from table 1 inner join table 2  
on table 1 . column name = table 2 . column name;
```

```
select Sname, cname from student  
inner join course on
```

```
student . cid = course . cid
```

Output:

Sname	Cname
-------	-------

Robo	C
------	---

chitti	Java
--------	------

Left outer join (or) Left join: This join returns all the records of table 1 and matching records of table 2. It returns NULL in the right side if there is no match.

Syntax:

select column names from table 1 left join table 2
on table 1.columnname = table 2.columnname;

Query:

select Sname, cname from students
left join course

on student.cid = course.cid

Output:

Sname	Cname
Robo	C
dana	NULL
chitti	Java

Right Outerjoin/Right Join: Right join returns all the records of right table (or) table 2 and matching records of left table (or) table 1.

It returns NULL in the left side when there is no match.

Syntax:

select column names from table 1 right join table 2
on table 1.columnname = table 2.columnname;

Query

select

right

on

Output

Sr

R

C

Full

the

rec

Syn

del

on

Qu

del

fu

or

Query:-

```
select Sname, cname from student
right join course
on student.cid = course.cid
```

Output:-

Sname	cname
Rebo	C
chitti	Java
NULL	C++

Full outer join / Full join : Returns all the records when there is a match in left table or right table records.

Syntax:-

```
select column names from table 1 fulljoin table 2,
on table 1 . columnname = table 2 . columnname ;
```

Query:-

```
select Sname, cname from student
full join course
on student.cid = course.cid
```

Output:-

Sname	Cname
Rebo	C
Sana	NULL
chitti	Java
Rebo	C
chitti	Java
NULL	C++

Group by clause:-

Group by clause is used to arrange the data into Groups. Group by clause uses select and was clauses to group the data.

* Group by clause uses a column to group the data.

* Group by uses aggregate functions.

Create table Employee (Sid number(10) PRIMARY KEY,

Fname Varchar(15),

Edepartment Varchar(15),

ESalary Number(10)

);

insert into Employee values (1, 'x', 'Admin', 20,000);

insert into Employee values (2, 'y', 'development', 50,000);

insert into Employee values (3, 'Z', 'testing', 30000);
 insert into Employee values (4, 'A', 'HR', 40000);
 insert into Employee values (5, 'B', 'HR', 30000);
 insert into Employee values (6, 'C', 'admin', 10000);

Output:-

eid	ename	edepartment	esalary
1	X	admin	20000
2	Y	development	30000
3	Z	testing	30000
4	A	HR	40000
5	B	HR	30000
6	C	admin	10000

Select column 1, column 2 from table where

condition GROUP BY column 1, column 2;

Query:-

Retrieve the total salary of employees by Grouping their departments.

⇒ select edepartment, sum(esalary) from Employee
 GROUP BY edepartment;

Output:-

edepartment	esalary
admin	30000
development	50000
testing	30000
HR	70000

* HAVING:-

• having clause is associated with groupby clause
having clause is used to specify some conditions
which is used to group the data by using groupby
clause.

Syntax:-

select column1, column2 from table where condition
GROUPBY column1, column2 HAVING condition;

⇒ select edepartment, sum(esalary) from Employee
GROUPBY edepartment HAVING esalary > 40000;

Output:-

edepartment	esalary
-------------	---------

development	50000
-------------	-------

Hr	70000.
----	--------